

© Joshua M. Willman 2020

J. M. Willman, *Beginning PyQt*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-5857-6_2

2. Getting Started with PyQt

Joshua M. Willman¹

(1) Hampton, VA, USA

Hello again! If this is your first time to ever make any kind of UI, then you might be wondering where to begin. Luckily in this chapter we are going to start you off with learning some of the fundamentals before jumping into the heavy lifting.

Creating any kind of UI can seem like a formidable task with all the different layouts, windows, and widgets there are to consider. Widgets are the buttons, menus, sliders, and other components that will make up our user interfaces. Therefore in every chapter, we are going to discuss how to build one or more projects and break them down into incremental steps and tasks in order to better help you understand the larger programs.

For example, in this chapter we are going to be looking at how to create a user profile GUI in PyQt. First, we will discuss what a user profile is generally comprised of and what kinds of widgets we will need for this project. Then, we will go step-by-step in learning how to build a basic window and add images and text to the GUI. Once it's all done, you'll have an application that looks like Figure [2-1](#).



Figure 2-1 User profile that displays your information for others to view

NoteFor those who already have a project in mind or just need to learn how to include certain widgets in their applications, many of the chap-

ters will include smaller programs focused specifically on how to code them.

Project 2.1 – User Profile GUI

A user profile refers to some kind of visual display used to present a specific user's personal data. The data on the profile helps to associate certain characteristics with that user and allows others to collect information about that individual. User profiles can be found on a number of different platforms including computer programs, online social networking sites, such as LinkedIn or Facebook, or operating systems. Depending upon what environment you are looking at, the appearance of the profile will change to fit the goals, rules, and needs of that application.

User profiles often have a number of parameters which are either mandatory or optional and allow for some level of customization to fit the preferences of the user, for example, a profile image or background colors. Many of them contain features which can typically be found in all types of profiles, such as the user's name and the "About" section to share some information about the user.

For this project, you will

1. Create an empty window in PyQt and find out
 - a. About the basic classes and modules needed to set up your GUI
 - b. How to modify the window size and title
2. Learn about creating widgets
 - a. Specifically `QLabel` to add text and images to your GUI
 - b. How to organize the widgets in your window using `move()`

Design the GUI Layout

Let's break down the user profile GUI and plan out what widgets you need and how they will be arranged in the window as seen in Figure [2-2](#).

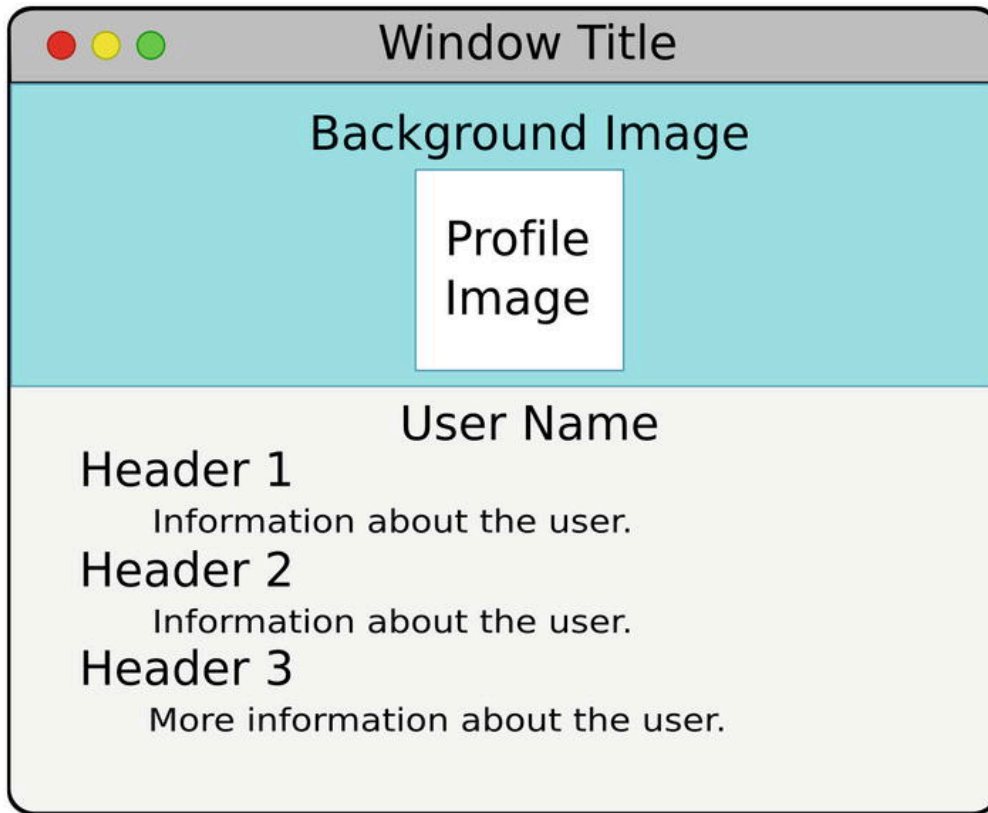


Figure 2-2 Layout for the user profile GUI

In real user profile applications, there are often a combination of different widgets. Some are interactive, allowing the viewer to click the links in the profile or “Like” buttons, while others are only meant to be read and cannot be altered by the viewer. In this chapter, we are looking at designing a basic interface in PyQt that shows a way to display information in the window using the **QLabel** widget.

The user interface can be divided into two parts:

1. The background image and profile image on the top.
2. And the user’s name and information on the bottom. The text on the bottom can further be broken down into smaller sections that are delineated by the use of different font sizes.

Create an Empty Window

A GUI application generally consists of a main window and possibly one or more **dialog boxes**. The main window in your program can consist of a menubar, a status bar, and other widgets, whereas a dialog is made up of buttons and is created to communicate information to the user and prompt them for input. An alert window that pops up asking you if you

want to save changes to your document is an example of a dialog. Dialog boxes will be covered further in Chapter 3.

Listing 2-1 walks you through the steps to create an empty GUI window.

```
# basic_window.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import QApplication, QWidget
class EmptyWindow(QWidget):
    def __init__(self):
        super().__init__() # create default constructor for QWidget
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setGeometry(100, 100, 400, 300)
        self.setWindowTitle('Empty Window in PyQt')
        self.show()
# Run program
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = EmptyWindow()
    sys.exit(app.exec_())
```

Listing 2-1 Create an empty window in PyQt

Your initial window should look similar to the one in Figure 2-3 depending upon your operating system.

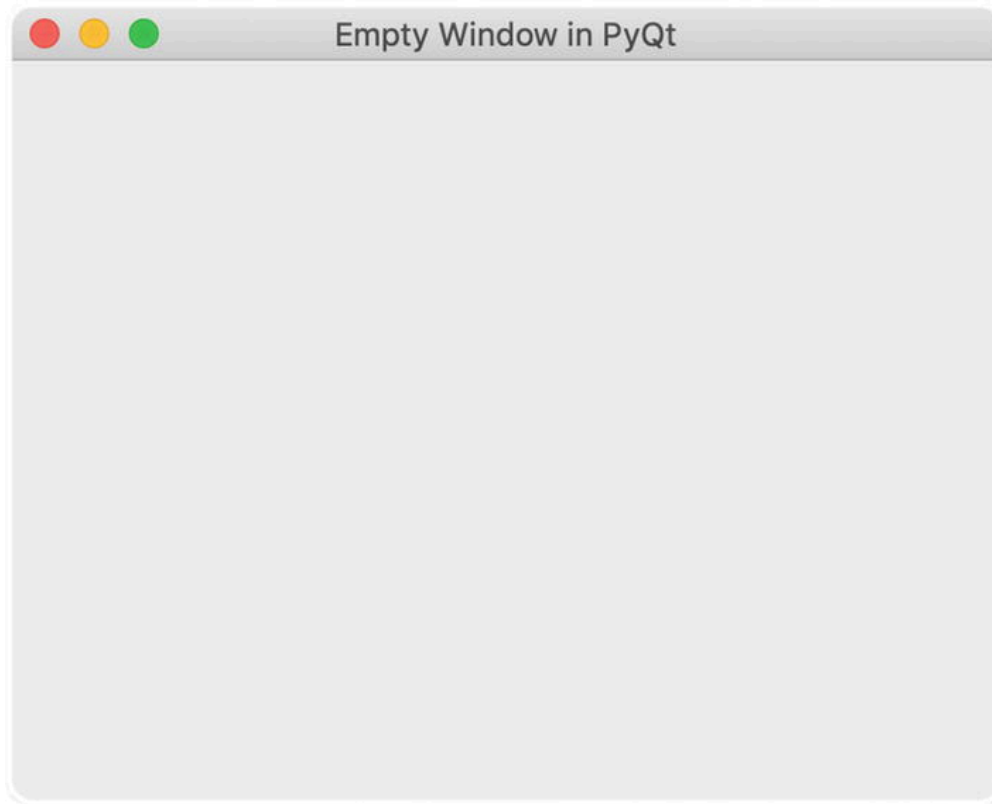


Figure 2-3 Empty window created with PyQt5

Explanation

Walking through the code, we first start by importing the `sys` and `PyQt5` modules that we need to create a window. We commonly use `sys` in order to pass command-line arguments to our applications and to close them.

The `QtWidgets` module provides a set of UI elements that can be used to create desktop-style GUIs. From the `QtWidgets` module, we import two classes, `QApplication` and `QWidget`. You only need to create a single instance of the `QApplication` class, which manages the application's main event loop, flow, initialization, and finalization, as well as session management. Take a quick look at

```
app = QApplication(sys.argv)
```

`QApplication` takes as an argument `sys.argv`. You can also pass in an empty list if you know that your program will not be taking any command-line arguments using

```
app = QApplication([])
```

Next we create a `window` object that inherits from the class we created, `EmptyWindow`. Our class actually inherits from `QWidget`, which is the base

class for which all other user interface objects are derived.

We need to call the `show()` method on the `window` object to display it to the screen. This is located inside the `initializeUI()` function in our `EmptyWindow` class. You will notice `app.exec_()` in the final line of the program. This function starts the event loop and will remain here until you quit the application. `sys.exit()` ensures a clean exit.

If all of this is a little confusing as to why we have to create an application before we create the window, think of `QApplication` as the frame that contains our window. The `window`, our GUI, is created using `QWidget`. Before we can create our GUI, we must create an instance of `QApplication` that we can place `window` in. Take a look at the following code to better see the order of creating a window in PyQt5 using procedural programming:

```
# 1. Import necessary modules
import sys # use sys to accept command-line arguments
from PyQt5.QtWidgets import QApplication, QWidget
app = QApplication(sys.argv) # 2. Create application object
window = QWidget() # 3. Create window
window.show() # 4. Call show to view GUI
sys.exit(app.exec_()) # Start the event loop and use sys.exit # to close the applicat
```

Modifying the Window

The preceding `EmptyWindow` class contains a function, `initializeUI()`, that creates the window based upon the parameters we specify. The `initializeUI()` function is reproduced as follows:

```
def initializeUI(self):
    """
    Initialize the window and display its contents to the screen.
    """
    self.setGeometry(100, 100, 400, 300)
    self.setWindowTitle('Empty Window in PyQt')
    self.show()
```

`setGeometry()` defines the location of the window on your computer screen and its dimensions, width and height. So the window we just started is located at `x=100, y=100` in the window and has `width=400` and `height=300`. `setWindowTitle()` is used to change the title of our window.

We will look at further customization of the window's layout in Chapter [4](#) and appearance in Chapter [6](#).

Note Throughout the book we will be looking at Python code written using **object-oriented programming (OOP)**. If you need a refresher on OOP, there is a quick guide that can be found in Appendix B.

The QLabel Widget

Now that we have a fundamental understanding of what it takes to create the window, we can move on and add more functionality with widgets using **QLabel**. A QLabel object acts as a noneditable placeholder to display text, images, or movies. It is also useful for creating labels around other widgets to specify their roles or give them titles. QLabel widgets can also display plain text, hyperlinks, or rich text.

In Listing [2-2](#) we are going to take a look at extending the ability of our window by showing how to create both text and image labels.

```
# labels.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import QApplication, QWidget, QLabel
from PyQt5.QtGui import QPixmap

class HelloWorldWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setGeometry(100, 100, 250, 250)
        self.setWindowTitle('QLabel Example')
        self.displayLabels()
        self.show()
    def displayLabels(self):
        """
        Display text and images using QLabels.
        Check to see if image files exist, if not throw an
```



```
exception.  
    """  
    text = QLabel(self)  
    text.setText("Hello")  
    text.move(105, 15)  
    image = "images/world.png"  
    try:  
        with open(image):  
            world_image = QLabel(self)  
            pixmap = QPixmap(image)  
            world_image.setPixmap(pixmap)  
            world_image.move(25, 40)  
    except FileNotFoundError:  
        print("Image not found.")  
# Run program  
if __name__ == '__main__':  
    app = QApplication(sys.argv)  
    window = HelloWorldWindow()  
    sys.exit(app.exec_())
```

Listing 2-2 Create an empty window in PyQt

Once you run the program, you should see a window similar to Figure [2-4](#).

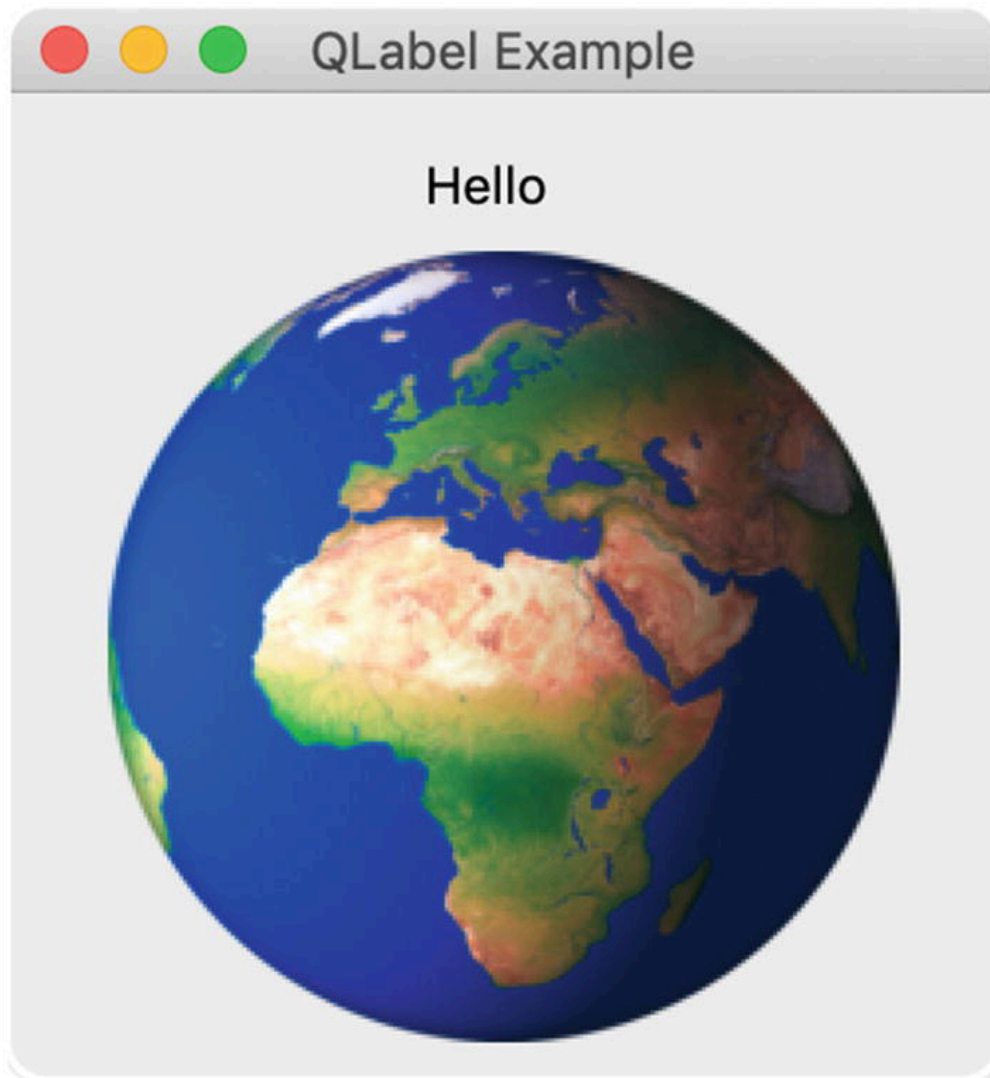


Figure 2-4 Example of using QLabel widgets to create images and text

Explanation

We begin again by first importing the necessary PyQt modules. To create the window, we need to import the `QtWidgets` class, and since this time we are going to be using the `QLabel` widget, we need to include it as well in our import statement.

This time we also need to import the `QtGui` module, as well. `QtGui` handles numerous graphical elements. `QPixmap` is a Qt class that is optimized for showing images on the screen.

We then go through a similar process of creating our application, creating a `HelloWorldWindow` class that inherits from the `QWidget` base class, initializing the size of the window with `setGeometry()` and the title of our GUI. Then we use the `show()` method to display the window and use

`exec_()` to begin the event loop. Finally, `sys.exit()` is used to close our program.

The `HelloWorldWindow` class contains a `displayLabels()` function that we will use to display text and images. First, you must create a `QLabel` object and specify what the label will say using `setText()`. Here the text is set to “Hello”. In the following line, we use the `move()` function to arrange the label in the window:

```
text = QLabel(self)
text.setText("Hello")
text.move(105, 15)
```

PyQt5 has a number of layout methods including horizontal layouts, grid layouts, as well as **absolute positioning**. For the programs created in this chapter, we will be using absolute positioning with the `move()` method. With `move()`, you only need to specify the x and y pixel values of the widget’s top-left corner to arrange it in the window. For our text label, we specify the values to be x=105 and y=15.

Our image is loaded in a similar fashion, creating a `QLabel` to be placed in the main window. Then we construct a `QPixmap` of the image and use `setPixmap()` to show the image displayed on the `world_image` label. The label’s absolute location is set using `move()`.

Each of PyQt’s different classes has their own methods that can be used to customize and change their look and functionality. In Appendix B, you can find a list of the widgets used in this book along with some of the more common methods you are likely to use to modify them.

User Profile GUI Solution

You have now learned many of the basic tools used to create the user profile GUI. This project is comprised of all `QLabel` widgets and its goal is to help you learn the fundamentals of creating GUIs in PyQt5. The `QLabel` widgets are used to display personal information specified by the user.

After you have followed along with Listing [2-3](#), it is encouraged to practice modifying the size of the window, add your own text or images, and practice

using other QLabel methods to see how all of the different parts work together to make the GUI window.

```
# user_profile.py
# Import necessary modules
import sys, os.path
from PyQt5.QtWidgets import QApplication, QLabel, QWidget
from PyQt5.QtGui import QFont, QPixmap

class UserProfile(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setGeometry(50, 50, 250, 400)
        self.setWindowTitle("2.1 - User Profile GUI")
        self.displayImages()
        self.displayUserInfo()
        self.show()
    def displayImages(self):
        """
        Display background and profile images.
        Check to see if image files exist, if not throw an exception.
        """
        background_image = "images/skyblue.png"
        profile_image = "images/profile_image.png"
        try:
            with open(background_image):
                background = QLabel(self)
                pixmap = QPixmap(background_image)
                background.setPixmap(pixmap)
        except FileNotFoundError:
            print("Image not found.")
        try:
            with open(profile_image):
                user_image = QLabel(self)
                pixmap = QPixmap(profile_image)
                user_image.setPixmap(pixmap)
                user_image.move(80, 20)
        except FileNotFoundError:
            print("Image not found.")
    def displayUserInfo(self):
        """
```

Create the labels to be displayed for the User Profile.

"""

```
user_name = QLabel(self)
user_name.setText("John Doe")
user_name.move(85, 140)
user_name.setFont(QFont('Arial', 20))

bio_title = QLabel(self)
bio_title.setText("Biography")
bio_title.move(15, 170)
bio_title.setFont(QFont('Arial', 17))
about = QLabel(self)
about.setText("I'm a Software Engineer with 8 years\
    experience creating awesome code.")
about.setWordWrap(True)
about.move(15, 190)
skills_title = QLabel(self)
skills_title.setText("Skills")
skills_title.move(15, 240)
skills_title.setFont(QFont('Arial', 17))
skills = QLabel(self)
skills.setText("Python | PHP | SQL | JavaScript")
skills.move(15, 260)
experience_title = QLabel(self)
experience_title.setText("Experience")
experience_title.move(15, 290)
experience_title.setFont(QFont('Arial', 17))
experience = QLabel(self)
experience.setText("Python Developer")
experience.move(15, 310)
dates = QLabel(self)
dates.setText("Mar 2011 - Present")
dates.move(15, 330)
dates.setFont(QFont('Arial', 10))
experience = QLabel(self)
experience.setText("Pizza Delivery Driver")
experience.move(15, 350)
dates = QLabel(self)
dates.setText("Aug 2015 - Dec 2017")
dates.move(15, 370)
dates.setFont(QFont('Arial', 10))
```

```
# Run program

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = UserProfile()
    sys.exit(app.exec_())
```

Listing 2-3 Code for the user profile GUI

Explanation

If you followed along in this chapter, then this project is just a much longer version containing a background image and more text labels. Images for the labels are loaded using Python `try-except` clauses.

A few new things here are as follows:

- We import the `QFont` class from the `QtGui` module, allowing us to modify the size and types of fonts in our application using `QLabel`'s `setFont()` method.
- Using `move()`, we are able to easily overlap images. Take a look at the `displayImages()` function to see how to do so using absolute positioning.

Summary

At this point, you should have a fundamental understanding for getting started in creating your own GUIs with PyQt5. We looked at setting up a basic window with `QApplication` and `QWidget` classes, made some simple modifications to the look of the window, learned how to create text and image labels with `QLabel`, and saw how to arrange them in the window using the `move()` method. In subsequent chapters, we will continue to learn about more widgets and classes and learn how to use them to fit the requirements of the applications we wish to create.

It is worth noting that this user profile is by no means complete. A user's profile is generally more interactive, including links, buttons, and menus. As you go through other chapters, you should come back and improve the user profile GUI as a means to apply what you have learned.

